Forza Fantasy League — Technical Overview (V2)

A multi-sport fantasy gaming platform. Web + native mobile, real-time scoring, social "Clubhouse" leagues, and a peer-to-peer coin economy.

*Document purpose: a concise, honest briefing for a technical stakeholder doing a **quick assessment** of what exists today, so they can give informed feedback. For full detail see the companion "Technical Documentation (V2)" and "Remediation Backlog V2". Reflects branch `v2` at 2026-06-30. Supersedes `TECH_OVERVIEW.md` (2026-06-26).*

> **If you are the reviewing developer:** §8 and §9 are written for you. §8 is an at-a-glance "what's solid / what's half-built / what's open" table; §9 lists the specific questions we'd value your feedback on.

---

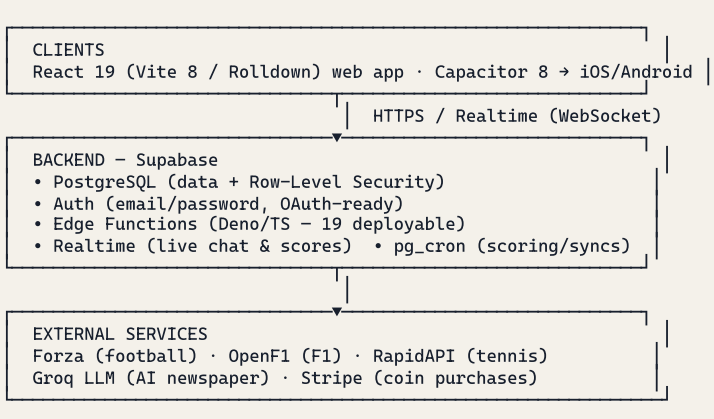
1. What the platform is

A fantasy-sports platform where friends create private leagues, draft or buy squads, compete on live match scoring, and engage through social features (Clubhouse, chat, an AI-generated league "newspaper", trading, prediction bets, a coin economy). It began as a football product (live, ~50-user pilot on `main`) and has been extended on the `v2` branch into a **clubhouse-centric multi-sport platform** covering **Football, Formula 1, and Tennis** with a shared social and competition layer.

Product type	Fantasy sports — web app + iOS/Android (Capacitor wrapper)
Sports	Football (live pilot), Formula 1, Tennis (both built + routed on `v2`)
Core loops	Squad building · live scoring · leagues & standings · drafts · trading · side bets · Clubhouse social feed
Monetisation	Virtual coin wallet + Stripe purchase path + P2P challenge betting (built on `v2`, not yet live)
Live status	Football live (~50-user pilot on `main`); F1/Tennis/coins/Clubhouse built on `v2` (not yet deployed — Week-12 merge gate)

---

2. Technology stack at a glance



```
Hosting: Vercel (frontend, auto-deploy from main) · Supabase (managed backend)
Local dev: Dockerfile + docker-compose (frontend + Postgres + Deno runner)
```

Layer	Technology
Frontend	React 19, Vite 8 (Rolldown), Tailwind CSS 4, React Router 7 (27 screens, all `React.lazy`)
Mobile	Capacitor 8 (single codebase → native iOS & Android shells)
Backend	Supabase: PostgreSQL, Auth, Deno Edge Functions, Realtime, pg_cron
Data feeds	Forza (football), OpenF1 (F1), RapidAPI (tennis) — behind a provider-adapter seam
AI	Groq (`llama-3.1`) for the generated league newspaper
Payments	Stripe (virtual coin packs)
Hosting / CI	Vercel (frontend) · GitHub Actions (security/lint/build/E2E)

3. Architecture in one paragraph

The client is a single-page React app (also wrapped natively for mobile via Capacitor) that talks directly to **Supabase**. Supabase provides the database, auth, and real-time channels. **Security-sensitive and money/points logic runs server-side** in PostgreSQL `SECURITY DEFINER` functions (RPCs) and Deno Edge Functions, with **Row-Level Security** isolating each user's data. **Scheduled jobs** (pg_cron) drive the live pipeline: sync fixtures/players from external sports APIs, ingest live events, and run an idempotent scoring engine that converts real-world performance into fantasy points. A **provider-adapter seam** (`_shared/providers/`) abstracts the data feeds behind a canonical model. The frontend auto-deploys to **Vercel** on merge to `main`; Edge Functions deploy manually behind a CI drift gate.

4. The core systems

System	What it does
Scoring engine	Live match stats → fantasy points per position; captain multipliers, auto-subs, per-round rules. Idempotent and re-runnable.
Squad & transfer engine	Server-side atomic transfers with budget, club-cap, formation, and transfer-window enforcement.
Draft system	Lottery and reverse-standings drafts, wishlists, knockout-phase keeps.
Leagues & competition	Classic, draft, head-to-head, cup formats; standings, ranks, gazette feed.
Trading & auctions	Player-for-player trades and two-phase auction listings.
Side bets	Commissioner-created prediction bets, auto-resolved against results.
Coin wallet (P2P)	Virtual-currency wallet with escrow, Stripe top-ups, audited ledger, challenge betting. **No cash-out path** (one-way value flow).

System	What it does
AI newspaper	A daily LLM-generated league "front page" with reactions and comments.
Multi-sport modules	F1 (paddocks, race scoring) and Tennis (player boxes, tournament rosters, ATP Finals pick'em).
Clubhouse (circle)	A cross-sport social container above leagues, with a shared feed and a (scaffolded) cross-sport meta-standing.

---

5. Data & live pipeline

1. **Sync** — scheduled jobs pull fixtures/players/statuses from the sports APIs.
2. **Ingest** — during matches, a job ingests live events and flips fixtures to "live".
3. **Score** — the scoring engine runs every 2 min for live fixtures, plus post-match and late-finisher passes.
4. **Settle** — once every fixture in a round finishes, the round is marked complete: standings update, H2H resolves, the feed is written, a recovery snapshot is stored.

Idempotent (safe to re-run) with overlapping schedules so a missed run is caught by the next.

---

6. Engineering practices

- **Source control:** GitHub, feature-branch + PR workflow; protected `main`. Two-branch model: `main` (live football pilot) + `v2` (the multi-sport asset).
- **CI:** GitHub Actions runs a **security gate** (`npm audit`, `madge --circular`, encoding scan, function-drift check) → lint → build → Playwright UI smoke, with E2E gated on the first three. `npm audit` is clean (0 vulns).
- **Containerization:** multi-stage `Dockerfile` + `docker-compose.yml` (frontend + local Postgres + Deno runner) for portable local dev.
- **Database:** versioned SQL migrations; append-only discipline; server-side business logic in `SECURITY DEFINER` functions. *(Caveat: migrations are hand-applied — no single reproducible schema baseline yet; see §8.)*
- **Security model:** RLS on user data; protected-column triggers; HMAC-verified service-role auth; client-immutable `is_admin`; constant-time Stripe verification; strong CSP/headers.
- **Provider abstraction:** a canonical data model + adapter registry (`forza` / `opta` / `manual`) so a new data feed is one adapter file.
- **Documentation:** an extensive in-repo knowledge base (architecture, API, deployment runbooks, scoring rules, a central TRACKER).

---

7. Scale & footprint

Metric	Value
Frontend code	~41,000 lines (React/JSX)
Edge Functions	21 (19 deployable, Deno/TypeScript)
Database migrations	243 SQL files
Screens / routes	~27 (football, F1, tennis, Clubhouse, social, admin)
Largest components	SquadScreen 2,219 ln · LeagueScreen 1,894 · CommissionerPanel 1,828
Current users	~50 (football pilot)

---

8. Honest state map (for the reviewing developer)

✅ Solid / done

- Phase-0 security closed: HMAC-verified service-role auth; all scoring functions gated; `is_admin` client-immutable; constant-time Stripe verification; DB-idempotent purchases; coin ledger with no cash-out path; protected-column triggers on squads & wallets.
- Idempotent scoring pipeline with auto-subs, captain reassignment, settled-round guards, and game-state recovery tables.
- CI security gate; 0 npm vulnerabilities; function-drift gate; 27 lazy-loaded screens; Node pinned; containerized local dev; strong CSP/headers.
- Multi-sport schema (sports/circles/trophy_ledger) + F1 (7 screens) + Tennis (7 screens) built and routed; clubhouse-centric IA.
- Provider-adapter seam (canonical model + registry; Opta stub ready for a buyer's feed).

◐ Half-built / scaffolded (works, but with a gap to close)

- **Provider independence:** the shared Forza client + canonical types exist, but the sync functions still parse raw Forza JSON inline and the DB spine is still `tournaments.forza_id` (rename to `provider_key` designed, not built). `sync-player-status` not yet migrated onto the shared client.
- **Cross-sport meta-standing:** `trophy_ledger` table + `get_circle_meta_standings()` RPC exist, but **no scoring path writes trophies** — the unified leaderboard is empty today.
- **Observability:** Sentry is integrated in code but `VITE_SENTRY_DSN` isn't set in Vercel — **no errors are actually captured in production yet**; edge functions have no error tracking; no failed-cron alerting.
- **God components:** improved (SquadScreen 2,879 → 2,219) but 5 files still exceed 1,400 lines with data + business logic + JSX interleaved.

☐ Open / not started

- **Reproducible schema baseline:** 243 migrations / 19 duplicate number prefixes, hand-applied, data fixes interleaved with DDL, no `schema.sql`. A clean environment cannot be rebuilt from the repo. *(Highest-leverage item.)*

- **Automated test coverage of money/game logic:** none. CI runs only a UI render smoke; 8 logic specs run manually against live prod data. No unit framework.
- **Operational DR:** single environment, no PITR, manual JSON backups, no staging gate.
- **Ownership-transfer hygiene:** project ref still in ~30 (mostly append-only) files; no transfer runbook; developer-machine PAT to rotate; Dockerfile pins Node 20 vs. engines-24.
- **No-cash-out as a positive schema constraint** (today it's enforced by absence); CSP `'unsafe-inline'`; F1 prediction bets readable via a broad RLS policy.

---

9. What we'd value your feedback on

If you have limited time, these are the questions where an external view is most useful:

1. **Data-layer reproducibility (DATA-1)** — is a `pg_dump --schema-only` baseline + archived history the right move, or would you adopt the Supabase migration framework / a tool like Atlas/sqitch from here? How would you sequence it without disrupting the live pilot?
2. **Test harness (TEST-1)** — given a local Postgres is now available via `docker-compose`, what's the fastest path to meaningful coverage of the hotspot RPCs (`execute_transfer_atomic`, `resolve_bet`, `set_lineup`, `calculate-scores`)? pgTAP vs. Deno integration vs. Vitest-against-local-PG?
3. **Provider abstraction finish (ARCH-2)** — is the canonical-model approach sound, and is the `forza_id` → `provider_key` additive rename the right level of effort, or over-engineering for the current sport set?
4. **Backend portability** — the whole runtime is Supabase primitives (Deno functions, pgcron, Supabase Auth/Realtime). For a buyer who must run on their own cloud, how would you frame the realistic re-platforming cost vs. keeping Supabase?
5. **Maintainability priorities** — would you prioritise a data-fetching layer (TanStack Query) and god-component decomposition, or incremental TypeScript, first?
6. **Anything that alarms you** — security, concurrency, data integrity, or operational risk we've under-weighted.

---

10. Maturity & roadmap (honest summary)

The platform is **feature-complete across three sports and functional in production for the football pilot**, with the deal-blocking security items closed and the major buyer-DD portability blockers (containerization, provider lock-in, project-ref hardcoding) substantially addressed since the prior review. The remaining work is the classic pilot-to-scale cluster: a reproducible/staged environment, automated test coverage of the money/scoring logic, observability activation, and finishing provider-independence + the cross-sport meta-standing. All are individually tractable and none require re-architecting the platform. Full detail and effort estimates are in the companion **Remediation Backlog V2**.

Companion documents: "Technical Documentation (V2)" (full detail) · "Remediation Backlog V2" (engineering roadmap). Last updated: 2026-06-30.